# Section 6.3 — CrewAI

## Role-Based Agent Teams for Structured Workflows

**Author:** Suresh D R | AI Product Developer & Technology Mentor

---

## Table of Contents


| # | Topic |
|---|---|
| 6.3.1 | Quick Recap — LangGraph and AutoGen and Their Limits |
| 6.3.2 | What Is CrewAI |
| 6.3.3 | LangGraph vs AutoGen vs CrewAI — When to Use Which |
| 6.3.4 | Core Components |
| 6.3.5 | How CrewAI Works — Step by Step |
| 6.3.6 | Real-World Use Cases That Require CrewAI |
| 6.3.7 | Process Types in CrewAI |
| 6.3.8 | When to Use CrewAI |
| 6.3.9 | Section Summary |


---

# 6.3.1 Quick Recap — LangGraph and AutoGen and Their Limits

Before learning CrewAI, it helps to understand what the other two frameworks do well — and where they fall short.

---

## LangGraph — Workflow Orchestration Through State

LangGraph connects agents through a shared state dictionary. Each agent reads from state, does its work, and writes back. The graph controls the order. Agents never talk to each other directly.

**LangGraph is great for:**

- Complex workflows with branching and conditional routing
- Workflows that must survive crashes (checkpointing)
- Workflows that need human approval at specific points
- Stateful, long-running pipelines

**Where LangGraph falls short:**

- Agents cannot debate or critique each other — they only read state
- No concept of roles or specialisation built in
- You must manually define every edge and routing logic
- Not ideal when agents need to work as a team with shared goals

**Real use cases that need LangGraph:**

*Patient Triage Routing* — A patient walks into emergency. The system collects symptoms, checks vitals, classifies severity (critical / urgent / non-urgent), and routes to the right ward. If a step fails midway, the system must resume from that step — not restart from admission. Branching + checkpointing = LangGraph.

*Insurance Claim Processing* — A health insurance claim arrives. The system verifies policy, checks documents, runs fraud detection, calculates payout, and triggers payment. If documents are missing it routes to a rejection branch. If fraud is detected it routes to manual review. 12 steps, must survive crashes, multiple branches = LangGraph.

*ICU Monitoring Agent* — Continuously monitors patient vitals every 5 minutes. If BP drops below threshold → alert nurse. If oxygen falls → alert doctor. If both critical → trigger emergency protocol. Real-time conditional routing based on state = LangGraph.

---

## AutoGen — Direct Agent-to-Agent Conversation

AutoGen lets agents talk to each other directly in a conversation. Every agent reads the full conversation history before responding. The GroupChatManager decides who speaks next.

**AutoGen is great for:**

- Debate and iterative refinement — Proposer vs Critic

- Tasks where agents need to directly respond to each other's arguments
- Code writing and execution loops
- Open-ended conversations where the number of turns is unpredictable

**Where AutoGen falls short:**

- No structured role assignment — agents can go off-topic
- No built-in task delegation — you cannot assign Task A to Agent 1 and Task B to Agent 2
- Conversations can go in circles without a clear outcome
- Hard to control when working with large teams of 5+ agents
- No concept of sequential task handoff — Agent 1 finishes, passes to Agent 2

**Real use cases that need AutoGen:**

*Treatment Plan Review* — A doctor proposes a treatment plan for a complex patient. A specialist from oncology challenges it. A pharmacist raises a drug interaction concern. The doctor revises. The specialist re-evaluates. This is a genuine back-and-forth between experts who must respond to each other's specific points — AutoGen.

*Hospital Budget Decision* — The Finance Head proposes cutting 10% from the nursing budget. The Nursing Director pushes back with patient safety data. The Operations Head suggests an alternative. The CFO reads all arguments and adjusts the proposal. Multiple experts debating a decision until consensus — AutoGen.

*Medical Research Debate* — Two researchers disagree on whether a new treatment protocol should be adopted. One argues efficacy data supports it. The other challenges the sample size and methodology. They debate through multiple rounds until one convinces the other or they agree on a conditional recommendation — AutoGen.

---

## The Gap Both Leave Open

Imagine you are building a hospital patient discharge planning system. You need:

- A **Medical Reviewer** who checks if the patient is clinically ready to go home
- A **Care Coordinator** who arranges post-discharge support and follow-ups
- A **Pharmacy Agent** who prepares the medication list and instructions
- A **Insurance Agent** who checks coverage and processes billing
- A **Report Writer** who assembles the final discharge summary

This is not a debate. Each agent has a specific job. Each job produces a specific output. The outputs flow in sequence — Medical Reviewer must finish before Care Coordinator can start. Care Coordinator must finish before Pharmacy Agent can prepare the list.

LangGraph could build this but requires you to manually wire every connection and write all the routing logic yourself.

AutoGen would let them debate — but this is not a debate. It is a structured team with defined roles and tasks.

**This is exactly what CrewAI was built for.**

---

## 6.3.2 What Is CrewAI

CrewAI is a framework for building **role-based agent teams** where each agent has a specific role, a goal, and a set of tools — and the team works together to complete a shared mission.

Think of CrewAI as building a **real work team** in code. Just like a real team has a manager, a researcher, an analyst, and a writer — each with a job description and deliverable — CrewAI agents have exactly that.

> **In one sentence:** CrewAI lets you build a team of AI specialists, assign each one a specific job, and orchestrate them to complete a complex mission together.

---

## The Core Idea

```
You define:
  WHO  — the agents (name, role, goal, backstory)
  WHAT — the tasks (description, expected output, which agent does it)
  HOW  — the process (sequential or hierarchical)

CrewAI handles:
  Task delegation
  Agent coordination
  Output passing between agents
  Tool calling
  Final result assembly
```

---

# 6.3.3 LangGraph vs AutoGen vs CrewAI — When to Use Which

This is the most important decision in Section 6.

---

## The Three Frameworks Side by Side

| Feature | LangGraph | AutoGen | CrewAI |
|---|---|---|---|
| How agents communicate | Through shared state | Direct messages | Task outputs passed between agents |
| Agent roles | You define manually | System prompts only | Built-in role, goal, backstory |
| Task assignment | Manual node wiring | GroupChat picks speaker | Explicit task-to-agent assignment |
| Workflow type | Graph with branches and loops | Open conversation | Sequential or hierarchical team |
| Debate and critique | Not built-in | Core strength | Not the focus |
| Structured team with clear jobs | Requires custom wiring | Hard to control | Core strength |
| Checkpointing / crash recovery | Built-in | Not available | Limited |
| Human approval gates | Built-in | Not built-in | Limited |

---

## The Hospital Example — Which Framework Fits?

**Scenario: Patient Discharge Planning**

```
Task: Prepare a complete discharge plan for a patient
      leaving the hospital after knee replacement surgery.

Subtasks:
  1. Medical Reviewer   — assess clinical readiness, check vitals, flag risks
  2. Care Coordinator   — arrange physiotherapy, home visit, follow-up appointment
  3. Pharmacy Agent     — prepare medication list, dosage, interaction warnings
  4. Insurance Agent    — verify coverage, raise billing, check approvals
  5. Report Writer      — compile everything into the final discharge summary
```

```
Flow: Task 1 must complete before Task 2 starts.
      Task 2 and Task 3 can run together.
      Task 5 needs outputs from all previous tasks.
```

**LangGraph:** you could build this but you would need to manually define 5 nodes, 8 edges, routing logic between them, and state fields for each output. A lot of wiring for a straightforward team task.

**AutoGen:** agents would debate and argue — but this is not a debate. A Pharmacy Agent does not need to argue with a Care Coordinator. They have separate jobs. AutoGen is the wrong tool.

**CrewAI:** define 5 agents with their roles, define 5 tasks with expected outputs, set process to sequential. CrewAI handles the rest. This is exactly what CrewAI was designed for.

---

## The Simple Decision Test

```
"Does this task need a branchable, resumable, stateful workflow?"
→ LangGraph

"Does this task need agents to debate and directly respond to each other?"
→ AutoGen

"Does this task need a structured team where each member has a clear job
 and outputs flow from one to the next?"
→ CrewAI
```

---

# 6.3.4 Core Components

CrewAI has four core building blocks.

---

## Component 1 — Agent

An Agent is a specialist with a defined identity. It has four key properties:

```
from crewai import Agent

medical_reviewer = Agent(
    role="Senior Medical Reviewer",
    goal=(
        "Assess whether the patient is clinically ready for discharge. "
        "Identify any medical risks that could cause readmission."
    ),
    backstory=(
        "You are a senior physician with 15 years of experience in hospital discharge planning. "
        "You are thorough, cautious, and always prioritise patient safety over efficiency."
    ),
    tools=[patient_records_tool, vitals_checker_tool],
    verbose=True
)
```

**role** — the job title. Shapes how the LLM approaches the task.

**goal** — what this agent is trying to achieve. Kept in mind for every decision.

**backstory** — the agent's expertise and personality. Makes the agent behave like a genuine specialist.

**tools** — what the agent can call (APIs, databases, calculators, search).

---

## Component 2 — Task

A Task is a specific piece of work assigned to a specific agent. It has a description, an expected output, and a reference to the agent who should do it.

```
from crewai import Task

review_task = Task(
    description=(
        "Review the discharge readiness of patient ID P-4821. "
        "Check vitals (BP, temperature, oxygen levels), wound status, and pain score. "
        "Identify any clinical risks that could lead to readmission within 30 days."
    ),
    expected_output=(
```

```
        "A clinical readiness report with: "
        "1. Discharge recommendation (ready / not ready / conditional) "
        "2. List of flagged risks with severity "
        "3. Any conditions to be met before discharge"
    ),
    agent=medical_reviewer
)
```

**description** — exactly what needs to be done. The more specific, the better the output.

**expected_output** — what the output should look like. CrewAI uses this to evaluate quality.

**agent** — which agent is responsible for this task.

---

## Component 3 — Crew

A Crew is the team — the collection of agents and tasks, with a defined process for how they work together.

```
from crewai import Crew, Process

discharge_crew = Crew(
    agents=[medical_reviewer, care_coordinator, pharmacy_agent, report_writer],
    tasks=[review_task, coordination_task, pharmacy_task, report_task],
    process=Process.sequential,   # tasks run one after another
    verbose=True
)
```

**agents** — the team members.

**tasks** — the work items, in the order they should be done.

**process** — Sequential (one after another) or Hierarchical (manager delegates to workers).

---

## Component 4 — Tools

Tools are functions the agent can call to interact with the real world. CrewAI uses the same tool pattern as LangChain.

```
from crewai_tools import tool

@tool("Patient Records Lookup")
def get_patient_records(patient_id: str) -> str:
    """Look up patient medical records by patient ID."""
    # connects to hospital database or EHR system
    return records_database.get(patient_id)

@tool("Vitals Checker")
def check_vitals(patient_id: str) -> str:
    """Retrieve latest vitals for a patient: BP, temperature, oxygen, heart rate."""
    return vitals_api.get_latest(patient_id)
```

The agent reads the tool description and decides when to call it — just like in LangGraph and AutoGen.

---

## 6.3.5 How CrewAI Works — Step by Step

Let us walk through exactly how the hospital discharge planning crew executes.

---

### The Setup

```
CREW: Hospital Discharge Planning Team
MISSION: Prepare a complete discharge plan for patient P-4821

AGENTS:
  Medical Reviewer   - checks clinical readiness
  Care Coordinator   - arranges post-discharge support
  Pharmacy Agent     - prepares medication plan
  Report Writer      - compiles the discharge summary

TASKS (sequential):
  Task 1 → Medical Reviewer
```

```
  Task 2 → Care Coordinator   (receives Task 1 output)
  Task 3 → Pharmacy Agent     (receives Task 1 and 2 output)
  Task 4 → Report Writer      (receives all previous outputs)
```

---

## Step-by-Step Execution

### Step 1 — Medical Reviewer runs Task 1

```
Medical Reviewer receives: Task description + patient ID
Medical Reviewer calls: get_patient_records("P-4821")
Medical Reviewer calls: check_vitals("P-4821")
Medical Reviewer reads: BP 128/82, temp 36.8, O2 98%, pain score 3/10
Medical Reviewer writes: Clinical Readiness Report

OUTPUT written by Medical Reviewer:
  Discharge Recommendation: CONDITIONAL
  Flagged risks: mild hypertension (BP slightly elevated)
  Conditions: BP must be below 130/85 before discharge
  Next step: re-check BP in 2 hours
```

### Step 2 — Care Coordinator receives Task 1 output and runs Task 2

```
Care Coordinator receives: Task description + Medical Reviewer output
  (knows patient has conditional discharge, BP monitoring needed)

Care Coordinator calls: schedule_physiotherapy("P-4821", "knee replacement")
Care Coordinator calls: arrange_home_visit("P-4821", "day 3 post discharge")
Care Coordinator calls: book_followup("P-4821", "ortho", "week 2")

OUTPUT written by Care Coordinator:
  Physiotherapy: booked Monday, Wednesday, Friday for 4 weeks
  Home visit: District nurse Day 3 post discharge
  Follow-up: Dr Sharma ortho clinic, 2 weeks post discharge
  Note: BP re-check result to be added before finalising
```

### Step 3 — Pharmacy Agent receives Task 1 and 2 outputs and runs Task 3

```
Pharmacy Agent receives: Task description + Medical Reviewer output + Care Coordinator output
  (knows about knee replacement, mild hypertension, discharge plan)

Pharmacy Agent calls: get_medication_list("P-4821")
Pharmacy Agent calls: check_interactions("ibuprofen", "amlodipine")

OUTPUT written by Pharmacy Agent:
  Medications:
    Paracetamol 500mg — every 6 hours for pain (14 days)
    Ibuprofen 400mg   — every 8 hours with food (7 days)
    Amlodipine 5mg    — once daily for BP (ongoing)
  Warning: Ibuprofen may mildly elevate BP — monitor closely
  Instruction: Take all medications with food
```

**Step 4 — Report Writer receives all outputs and runs Task 4**

```
Report Writer receives: All three previous outputs

Report Writer compiles the final discharge summary:

PATIENT DISCHARGE SUMMARY — P-4821
========================================
Name         : [Patient Name]
Procedure    : Right knee replacement
Date         : [Today]
Status       : Conditional discharge (BP cleared at 2-hour recheck)

CLINICAL STATUS:
  Vitals stable. BP 128/82 → 126/80 at recheck. Cleared for discharge.

POST-DISCHARGE CARE:
  Physiotherapy: Mon/Wed/Fri for 4 weeks
  Home visit: District nurse Day 3
  Follow-up: Dr Sharma, 2 weeks

MEDICATIONS:
  Paracetamol, Ibuprofen, Amlodipine (see pharmacy sheet)
  Monitor BP daily due to Ibuprofen interaction.
```

```
REPORT COMPLETE. Ready for doctor signature.
```

---

## What CrewAI Did Automatically

```
1. Assigned Task 1 to Medical Reviewer
2. Passed Task 1 output to Care Coordinator for Task 2
3. Passed Task 1 + Task 2 outputs to Pharmacy Agent for Task 3
4. Passed all three outputs to Report Writer for Task 4
5. Assembled the final result

You did NOT need to:
  - Define state fields manually
  - Write routing functions
  - Wire edges between nodes
  - Handle output passing between agents
  CrewAI did all of this automatically.
```

---

# 6.3.6 Real-World Use Cases That Require CrewAI

These are scenarios where a structured team with clear roles and sequential task handoff is the right approach.

---

## Use Case 1 — Hospital Patient Discharge Planning

**Scenario:** Every patient leaving a hospital after surgery needs a discharge plan — clinical sign-off, care coordination, medication instructions, and billing sorted before they leave.

**Why CrewAI:**
Each department has a specific job. The clinical team must sign off before the care team can arrange support. The pharmacy cannot finalise the medication list until the clinical team confirms what procedures were done. The billing team cannot raise an invoice until the care plan is set. This is a structured sequential team — not a debate.

**The crew:**

| Agent | Role | Task |
|---|---|---|
| Medical Reviewer | Senior physician | Assess discharge readiness, flag risks |
| Care Coordinator | Discharge coordinator | Arrange physiotherapy, home visits, follow-ups |
| Pharmacy Agent | Clinical pharmacist | Prepare medication list with interactions |
| Insurance Agent | Billing executive | Verify coverage, raise invoice |
| Report Writer | Medical records officer | Compile final discharge summary |

---

## Use Case 2 — Clinical Trial Report Generation

**Scenario:** A pharmaceutical company needs to generate a regulatory submission report for a new drug trial. Data from multiple departments must be gathered, analysed, summarised, and reviewed before submission.

**Why CrewAI:**
Different specialists handle different sections. A biostatistician analyses the efficacy data. A safety reviewer summarises adverse events. A medical writer drafts the narrative. A regulatory expert checks compliance. The output of each feeds into the next — clearly sequential, clearly role-based.

**The crew:**

| Agent | Role | Task |
|---|---|---|
| Data Analyst | Biostatistician | Analyse trial data, calculate efficacy and safety metrics |
| Safety Reviewer | Pharmacovigilance expert | Summarise adverse events, assign causality |
| Medical Writer | Clinical writer | Draft the narrative summary sections |
| Regulatory Expert | Regulatory affairs | Check compliance with submission guidelines |
| Quality Reviewer | QA officer | Final review before submission |

---

## Use Case 3 — Patient Appointment and Diagnostic Workflow

**Scenario:** A patient books an appointment. The system must check insurance eligibility, assign the right specialist, prepare pre-consultation materials, and send reminders — all before the patient arrives.

**Why CrewAI:**
Each step depends on the previous one. Insurance must be checked before a specialist is assigned. The specialist must be assigned before pre-consultation materials can be prepared. Materials must be ready before reminders are sent. Sequential, role-based, no debate needed.

**The crew:**

| Agent | Role | Task |
|---|---|---|
| Insurance Agent | Eligibility checker | Verify patient coverage and co-pay |
| Scheduling Agent | Appointment coordinator | Match patient to right specialist and slot |
| Preparation Agent | Pre-consultation specialist | Prepare patient history summary for doctor |
| Communication Agent | Patient liaison | Send confirmation, reminders, and instructions |

---

## Use Case 4 — Research Paper Writing

**Scenario:** A medical research team needs to produce a paper on a new treatment approach. Different team members handle literature review, data analysis, writing, and peer review — each depending on the previous one's output.

**Why CrewAI:**
A researcher cannot write the paper before the literature review is done. An analyst cannot interpret data before it is collected and cleaned. A writer cannot draft the paper before the analysis is complete. A reviewer cannot critique before there is a draft. Pure sequential team handoff.

**The crew:**

| Agent | Role | Task |
|---|---|---|
| Literature Researcher | Research specialist | Search, filter, and summarise relevant papers |
| Data Analyst | Medical statistician | Analyse patient data, produce charts and statistics |
| Medical Writer | Scientific writer | Draft the paper using research and analysis |
| Peer Reviewer | Domain expert | Critique the draft, flag gaps and errors |
| Editor | Medical editor | Final polish, formatting, reference check |

## Use Case 5 — Hospital Staff Scheduling

**Scenario:** Every week, a hospital needs to create a shift schedule for 200 nurses across 8 departments — accounting for leave requests, skill requirements, regulatory staffing ratios, and department needs.

**Why CrewAI:**
The leave checker must first identify who is unavailable. The skill matcher must then identify who can cover which department. The compliance checker must verify minimum staffing ratios are met. The schedule builder creates the schedule. The notifier sends it out. Each step depends on the last.

**The crew:**

| Agent | Role | Task |
|---|---|---|
| Leave Checker | HR analyst | Identify unavailable staff for the week |
| Skill Matcher | Staffing coordinator | Match available nurses to required departments |
| Compliance Checker | Regulatory officer | Verify staffing ratios meet regulatory minimums |
| Schedule Builder | Operations coordinator | Build the weekly shift schedule |
| Notification Agent | Communications officer | Send schedule to all staff |

---

# 6.3.7 Process Types in CrewAI

CrewAI supports two ways of organising how agents work together.

---

## Sequential Process

Tasks run one after another in the order you define. The output of Task 1 automatically becomes context for Task 2. Task 2 output flows into Task 3, and so on.

```
Task 1  →  Task 2  →  Task 3  →  Task 4  →  Final Output
```

**When to use:** when each task clearly depends on the previous one. Hospital discharge, report writing, content review pipelines.

```
crew = Crew(
    agents=[...],
    tasks=[task1, task2, task3, task4],
    process=Process.sequential
)
```

---

## Hierarchical Process

A manager agent receives the overall mission and delegates subtasks to specialist agents. The manager reviews outputs and can reassign if quality is poor.

```
                Manager Agent
               /      |      \
          Task A    Task B    Task C
          (Agent 1) (Agent 2) (Agent 3)
               \      |      /
             Manager reviews and assembles
```

**When to use:** when a senior decision-maker needs to coordinate a team, quality-check outputs, and make final decisions. Clinical director overseeing multiple department reports.

```
crew = Crew(
    agents=[manager, specialist_1, specialist_2, specialist_3],
    tasks=[task_a, task_b, task_c],
    process=Process.hierarchical,
    manager_llm=llm_config
)
```

---

# 6.3.8 When to Use CrewAI

## Use CrewAI When

```
Your task is a team effort:
  Multiple specialists each contributing a distinct piece of work.
  Hospital discharge: clinical + care + pharmacy + billing + report.

Your tasks have a natural sequence:
  Each task produces output that the next task needs.
  Literature review → Analysis → Writing → Review → Edit.

Each role is clearly defined:
  You can describe each team member's job in one sentence.
  "The Pharmacy Agent prepares the medication list."

You want simple, clean orchestration:
  You do not want to manually wire state fields, edges, and routing.
  You want to define WHO does WHAT and let CrewAI handle the rest.

You need tool access per agent:
  Each specialist uses different tools relevant to their job.
  Medical Reviewer uses EHR lookup. Pharmacy Agent uses drug interaction checker.
```

## Do NOT Use CrewAI When

```
Agents need to debate and challenge each other:
  Use AutoGen. CrewAI agents do not argue back and forth.

You need complex conditional branching:
  Use LangGraph. CrewAI process is sequential or hierarchical, not branching.

You need crash recovery and checkpointing:
  Use LangGraph. CrewAI does not checkpoint state after every step.

Human approval is needed at specific points:
  Use LangGraph with interrupt_before.
```

---

# 6.3.9 Section Summary

CrewAI is the right tool when your task looks like a real work team — specialists with defined roles, working in a defined order, each producing a specific output that feeds the next person.

---

## The Three Frameworks at a Glance

| Framework | Best Metaphor | Core Strength | Avoid When |
|---|---|---|---|
| LangGraph | Factory assembly line with sensors | Branching, checkpointing, state management | Task is a team with clear roles |
| AutoGen | Boardroom debate | Direct agent-to-agent debate, iterative refinement | Task is structured with clear jobs |
| CrewAI | Project team with a brief | Role-based team, sequential task handoff | Need debate or complex branching |

---

## The Four Core Components

| Component | What It Is |
|---|---|
| Agent | A specialist with a role, goal, backstory, and tools |
| Task | A specific piece of work assigned to a specific agent |
| Crew | The team — agents + tasks + process type |
| Tools | Functions agents call to interact with real-world systems |

---

## The Key Rule

> **In CrewAI, you define WHO does WHAT.**
>
> CrewAI handles the rest — task delegation, output passing, coordination, and assembly.

---